

Supongamos el siguiente código que se ejecuta en 0,418s. Las etiquetas "V" y "bucle" están alineadas a 1MB y sabemos que todas las instrucciones x86 se codifican usando entre 1 y 12 bytes:

```
    movl $1000000, %ecx
    movl $0, %eax
bucle: addl V(,%ecx,4), %eax
    decl %ecx
    jnz bucle
```

b) **Calcula** el número de accesos físicos a memoria principal producidos al ejecutar el bucle.

```
Instrucciones ejecutadas = 3 * 1000000 = 3000000
Accesos a memoria = 4 (3 ins + 1 datos) * 2 (1 tabla páginas + 1 @física) * 1000000 =
8000000 accesos
```

c) **¿Cuál** es el tiempo de acceso a la memoria principal?

```
Ciclos programa = 3000000 * 6 + 8000000 * (ta-1) = 418000000 = 18000000 + 50*8000000
-> ta -1 = 50 -> ta = 51 ciclos
```

Vista la ineficiencia decidimos utilizar 2 caches (y 2 TLBs) separados de instrucciones y datos. Cada TLB tiene 8 entradas, la cache de datos es la original (4-asociativa de 16 KBytes de capacidad con bloques de 64 bytes) y la cache de instrucciones es de la misma capacidad y tamaño de línea pero de acceso directo. Las páginas del sistema son de 8 KB.

d) **¿Cuántos** fallos de cache de instrucciones y datos dará en total el código anterior?

```
2 fallos de instrucciones
(un fallo las instrucciones antes del bucle y otro las instrucciones del bucle)

1000000 / 16 de datos = 62502 fallos de datos
```

e) **¿Cuántos** fallos de TLB de datos dará en total el código anterior?

```
1000000 / 2048 -> 489 fallos de TLB
```

[illegible][illegible]

Problema 2. (3.2 puntos)

Una **CPU x86** está conectada a una cache de instrucciones (**\$I**) y una cache de datos (**\$D**). El conjunto formado por **CPU+\$I+\$D** esta conectado a una memoria principal formada por un único módulo DIMM estándar de 16 GBytes. Este DIMM tiene 8 chips de memoria **DDR-SDRAM (Double Data Rate Synchronous DRAM)** de 1 byte de ancho cada uno. La DDR-SDRAM tiene 16 bancos. El DIMM esta configurado para leer/escribir ráfagas de 64 bytes (justo el tamaño de bloque de las caches). La latencia de fila es de 3 ciclos, la latencia de columna de 4 ciclos y la latencia de precarga de 2 ciclos. Es posible que el conjunto **CPU+\$I+\$D** solicite múltiples bloques a la DDR (por ejemplo porque se produzca un fallo simultáneamente en **\$I** y en **\$D**). El controlador de memoria envía los comandos necesarios a la DDR-SDRAM de forma que los bloques sean transferidos lo más rápidamente posible y se maximice el ancho de banda. Los 24 bits de direcciones físicas se han mapeado de la siguiente forma:

Fila	Banco	Columna	Chip
23:16	15:12	11:3	2:0

Dadas las siguientes direcciones físicas de bloque:

A: 0xaa0100

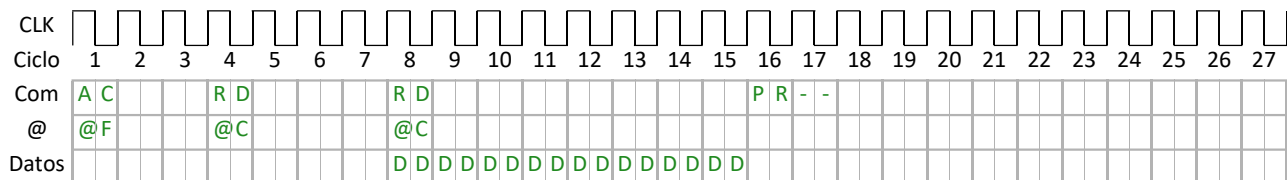
B: 0xaa0200

C: 0xaa1b00

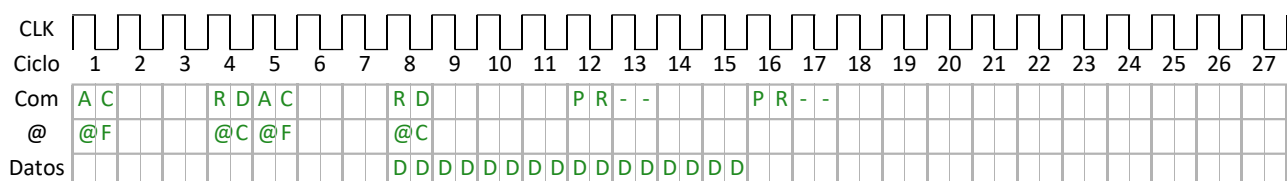
D: 0xbb1580

Rellena los siguientes cronogramas para la lectura de varios bloques de 64 bytes en función de la ubicación de los bloques involucrados de forma que se minimice el tiempo total. Indica la ocupación de los distintos recursos de la memoria DDR: bus de datos, bus de direcciones y bus de comandos. En todos los cronogramas supondremos que no hay ninguna página de DRAM abierta previamente.

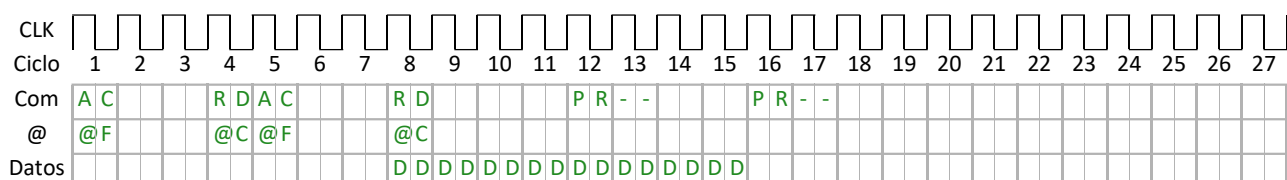
a) **Rellena** el siguiente cronograma para la lectura de los bloques A y B.



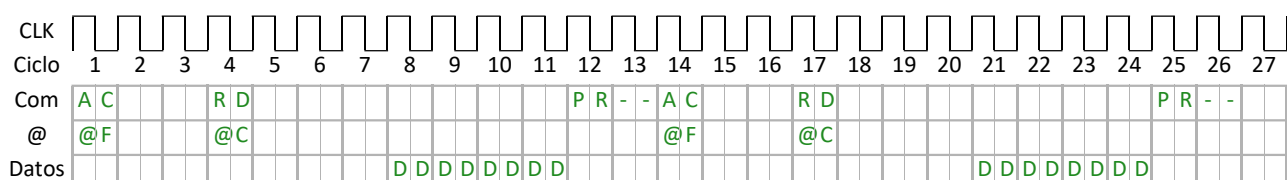
b) **Rellena** el siguiente cronograma para la lectura de los bloques A y C.



c) **Rellena** el siguiente cronograma para la lectura de los bloques A y D.



d) **Rellena** el siguiente cronograma para la lectura de los bloques C y D



Queremos estudiar el efecto de la búsqueda y decodificación de instrucciones en este procesador x86 con un conocido *benchmark*. Dicho procesador es un procesador segmentado, superescalar y con ejecución fuera de orden. En los procesadores segmentados pueden transcurrir decenas de ciclos hasta que se calcula la dirección destino del salto y el procesador decide si el salto se producirá (*taken*) o por el contrario se seguirá ejecutando en secuencia (*not taken*). Hemos obtenido que el 20% de las instrucciones dinámicas son saltos, y que el 75% de los saltos son *taken*. También hemos obtenido que, en un procesador ideal donde los saltos no penalizan, el CPI es de 0,3 ciclos/instrucción.

El procesador tiene un predictor de saltos sencillo que predice todos los saltos como *not taken*. Los saltos *not taken* no incurrir en ninguna penalización. Sin embargo, los saltos *taken* incurrir en una penalización de 18 ciclos.

e) **Calcula** el CPI del procesador con predictor de saltos sencillo.

$$\text{CPI} = 0,3 \text{ c/i} + 0,2 \text{ saltos/ins} * 0,75 \text{ taken/salto} * 18 \text{ ciclos/taken} = 3 \text{ ciclos/instrucción}$$

Ante la pérdida de rendimiento debida a los saltos, se ha decidido incorporar un predictor de saltos mas sofisticado. Este predictor tiene una tasa de aciertos del 97,5%. La penalización por fallo de predicción sigue siendo de 18 ciclos.

f) **Calcula** el CPI del procesador con predictor de saltos sofisticado.

$$\text{CPI} = 0,3 \text{ c/i} + 0,2 \text{ saltos/ins} * 0,025 \text{ fallos/salto} * 18 \text{ ciclos/fallo} = 0,39 \text{ ciclos/instrucción}$$

Los procesadores superescalares leen varias instrucciones cada vez que acceden a la cache de instrucciones, por lo que el número de accesos a la cache es menor que el numero de instrucciones ejecutadas. Sabemos que en el *benchmark* se ejecutan 10×10^9 instrucciones dinámicas y que se realizan 4×10^9 accesos a la cache de instrucciones, es decir, se realizan sólo 0,4 accesos a la cache por instrucción. Nuestro procesador traduce las instrucciones x86 a microoperaciones RISC (uops). Cada acceso a la cache consume 2,5 nJ (sea acierto o fallo) y se consumen 2 nJ por cada instrucción x86 descodificada y traducida a uops.

g) **Calcula** la energía consumida por la búsqueda y traducción de las instrucciones x86.

$$E = 10 \times 10^9 \text{ instr} * 2 \times 10^{-9} \text{ J/instr} + 4 \times 10^9 \text{ accesos} * 2,5 \times 10^{-9} \text{ J/accesos} = 30 \text{ J}$$

Se ha decidido introducir una cache de micro-ops (**uops**) donde se guardan las uops que ya han sido traducidas para que en caso de acierto no sea necesario descodificar y traducir de nuevo las instrucciones x86. Se ha medido que se generan 15×10^9 uops dinámicas y se realizan 4×10^9 accesos a la cache de uops. Gracias a esto, el número de accesos a la cache de instrucciones se ha reducido a $0,4 \times 10^9$ accesos y el número de instrucciones x86 descodificadas se ha reducido a 1×10^9 instrucciones. Un acceso a la cache de uops consume 1,75 nJ.

h) **Calcula** la energía consumida por el sistema con cache de uops.

$$E = 1 \times 10^9 \text{ instr} * 2 \times 10^{-9} \text{ J/instr} + 0,4 \times 10^9 \text{ accesos} * 2,5 \times 10^{-9} \text{ J/acceso} + 4 \times 10^9 \text{ accesos} * 1,75 \times 10^{-9} \text{ J/acceso} = 10 \text{ J}$$

[illegible][illegible]

Problema 3. (3.4 puntos)

La empresa **Can60**, dispone de un servidor formado por los componentes mostrados en la tabla siguiente. La tabla también muestra el número de componentes de cada tipo y el tiempo medio hasta fallo (MTTF) de cada componente:

Componente	Fuente alimentación	CPU	Placa base	DIMMs	Disco duro
Nº	1	1	1	4	1
MTTF (horas)	200.000	1.000.000	200.000	1.000.000	100.000

a) **Calcula** el MTTF del sistema suponiendo que este falla si falla alguno de los componentes.

$$1/\text{MTTF} = 1/2\text{e}5 + 1/1\text{e}6 + 1/2\text{e}5 + 4/1\text{e}6 + 1/\text{e}5 = 1/40000$$
$$\text{MTTF} = 40000 \text{ horas}$$

El disco duro del servidor es de tipo: **disco1**, pero la empresa posee inventariados en uno de sus ‘desordenados’ almacenes 23 discos más. En total, contando con el **disco1** del servidor, disponemos de 24 discos físicos: la mitad son de tipo: **disco1** y la otra mitad de tipo: **disco2**. El **disco1** inventariado posee un ancho de banda efectivo de 250 MBytes/s. El ancho de banda efectivo para un **disco2** es de 200 MBytes/s. Todos los discos poseen la misma capacidad. El MTTR de un disco físico cualquiera es de 10 horas.

En este servidor base (1 **CPU** + 1 **disco1**) ejecutamos una aplicación A que consta de dos fases bien diferenciadas:

- **Cálculo:** donde se usa exclusivamente la CPU, y es parcialmente paralelizable. Esta fase supone el 80% del tiempo de ejecución.
- **Entrada/Salida (E/S):** que prácticamente no usa la CPU, y pasa todo el tiempo accediendo al sistema de almacenamiento para leer y escribir datos. Sabemos que 3/5 del tiempo se realizan lecturas secuenciales, y 2/5 del tiempo se realizan escrituras aleatorias.

Para mejorar el rendimiento y fiabilidad de la parte de E/S de la aplicación A, nos planteamos sustituir el disco1 por alguna configuración de discos lógicos RAID. En base a las características de los dos tipos de discos y la aplicación A, hemos decidido probar las siguientes configuraciones:

- un RAID 5 con los discos1 (discos con mayor ancho de banda efectivo).
- un RAID 6 con los discos2 (configuración con más fiabilidad).

Del RAID 6 con los discos2, sabemos que el MTTF de esta configuración es un valor que está entre 70×10^6 y 80×10^6 horas.

b) **Calcula** el MTTF de un RAID 5 con los discos1. **Justifica** tu respuesta indicando la expresión matemática que utilizas para realizar el cálculo.

$$MTTF = MTTF_d^2 / (N * (N-1) * MTTR) = 100.000^2 / 12 * 11 * 10 = 7.575.757 \text{ horas} \rightarrow (\text{peor fiabilidad que RAID6})$$

Para acabar de decidirnos, evaluaremos algunas características más de las dos configuraciones.

Consideraremos las siguientes definiciones:

- **Ndi**: número de discos físicos del tipo i (Nd1 para disco1, Nd2 para disco2).
- **% útil**: el porcentaje de la capacidad total que es capacidad útil.
- **BW RD Secuencial**: ancho de banda efectivo para el caso de accesos de lectura secuenciales.
- **BW WR Aleatoria**: ancho de banda efectivo para el caso de accesos de escritura aleatorios.

c) **Completa** la siguiente tabla, y **justifica** con una expresión matemática cada uno de los cálculos que has realizado.

	RAID 5 Nd1 = 12	expresión	RAID 6 Nd2 = 12	expresión
% útil	91,6%	$(Nd1 - 1)/Nd1 * 100$	83,3%	$(Nd2 - 2)/Nd2 * 100$
BW RD Secuencial	3 GB/s	$250MB/s * Nd1$	2,4 GB/s	$200MB/s * Nd2$
BW WR Aleatoria	750MB/s	$250MB/s * (Nd1/4)$	400MB/s	$200MB/s * (Nd2/6)$

Elegimos sustituir el disco1 del servidor base (1 CPU + 1 disco1) por la configuración RAID 5 con 12 discos1. Llamaremos sistema E/S_RAID al formado por este RAID 5 (1 CPU + 1 RAID 5).

d) **Calcula** la ganancia en tiempo (speed-up) de toda la parte de E/S respecto del servidor base al ejecutar la aplicación A con el nuevo sistema E/S_RAID.

3/5 son lecturas secuenciales (0.6) y 2/5 son escrituras aleatorias (0.4)

$gR = 12$ y $gW = 3$

$Ses_RAID = 1/(0.6/gR + 0.4/gW) = 1/(11/60) = 60/11 = 5.45$

Al ejecutar A en el servidor base, la fase de Cálculo, la única donde se realizan cálculos en coma flotante, realiza 250×10^{12} operaciones en coma flotante, y la velocidad de ejecución es de 5 GFLOPS. La fase de Cálculo se divide en dos partes:

- una zona secuencial que corresponde al 2% del tiempo total de ejecución, y que no puede paralelizarse.
- una zona completamente paralelizable que corresponde al 98% del tiempo total de ejecución.

Para mejorar el rendimiento de la parte de Cálculo que es paralelizable, decidimos sustituir la única CPU del servidor base por 8 CPUs. Hemos visto que no es posible realizar una paralelización perfecta debido al tiempo de sincronización entre los distintos procesos. Se calcula que este tiempo de sincronización tiene una componente de penalización fija de 425 segundos, y otra penalización variable de 350 segundos por cada CPU que se añade al sistema.

e) **Calcula** el tiempo de ejecución de la fase de Cálculo, teniendo en cuenta la sincronización, utilizando 8 CPUs.

Tcalculo (1 CPU) = $250 \times 10^{12} / 5000 \times 10^6 = 50000$ segundos

Tcalculo (8 CPUs) = $50000 \times 0.02 + (50000 \times 0.98)/8 + 425 + 350(8-1) = 10000$ segundos ---> Scalculo = 5

Finalmente la configuración seleccionada para el servidor consiste en un multiprocesador con el número óptimo de CPUs y un sistema de almacenamiento con 4 discos SSD. Respecto del servidor base (1 CPU + 1 disco1) hemos obtenido ganancias de 6.4 para la fase de E/S y de 5.3 para la fase de cálculo al ejecutar A en la nueva configuración de servidor. La aplicación tiene 10^5 instrucciones estáticas y ejecuta 500×10^{12} instrucciones dinámicas.

f) **Calcula** los MIPS al ejecutar la aplicación en el servidor base. **Calcula** los MIPS y MFLOPS al ejecutar la aplicación en el servidor del multiprocesador con número óptimo de CPUs + sistema de E/S con 4 SSD.

servidor base: $T_{exe} = 50000/0.8 = 62500$ segundos

$MIPS = 500 \times 10^{12} \text{ instr} / 62.5 \times 10^3 \text{ segundos} * 1M/10^6 = 8000$ MIPS

servidor n_optimoCPUs y 4 SSD: Scalculo = 5.3 y $Ses_RAID_nuevo = 6.4$ ---> $S = 1/(0.8/5.3 + 0.2/6.4) = 5.5$

$MIPS = 8000 \text{ MIPS} \times 5.5 = 44000$ MIPS

$MFLOPS = (250 \times 10^{12} / 62.5 \times 10^3 \text{ segundos} * 1M/10^6) \times 5.5 = 4000 \text{ MFLOPS} \times 5.5 = 22 \text{ GFLOPS}$